PROJECT REPORT

Technical Documentation

RAG-Based Customer Support Assistant

with LangGraph & Human-in-the-Loop

Name: Om Dongare

Intern ID: IN226023402

Date: 24/04/2026

Document Type: Technical Documentation

Contents

1 Introduction 3
1.1 WhatisRAG? .... 1.2 . . . . . .
WhyisRAGNeeded? 1.3 . . . . . .
UseCaseOverview . . . . . .

2 SystemArchitectureExplanation 5
2.1 System Architecture Overview . . . . . .
2.2 Five Architecture Layers . . . . . .
2.2.1 Layer 1: User Interface . . . . . .
2.2.2Layer2:WorkflowOrchestration(LangGraph). . . . . .
2.2.3 Layer 3: AI Processing . . . . . .
2.2.4 Layer 4: Data Storage . . . . . .
2.2.5Layer5:HITLSystem . . . . . .

3 DesignDecisions 7
3.1 Chunk Size Choice: 500 Characters . . . . . .
3.2 . Embedding Strategy . . . . . .
3.3 Retrieval Approach: K=3, Cosine . . . . . .
3.4 Similarity PromptDesignLogic . . . . . .

4 WorkflowExplanation 9
4.1 LangGraph Usage . . . . . .
4.2 Node Responsibilities . . . . . .
4.3 State Transitions . . . . . .
4.3.1 NormalFlow(FAQQuery).. . . . . .
4.3.2 EscalationFlow(Complaint). . . . . .
4.4 LangGraph Workflow Diagram . . . . . .

5 ConditionalLogic 11
5.1 Intent Detection . . . . . .
5.2 Routing Decisions . . . . . .
5.3 . Decision Tree . . . . . .

6 HITLImplementation 13
6.1 Role of Human Intervention . . . . . .
6.2 Benefits and Limitations of HITL . . . . . .
6.3 HITL Escalation Flow . . . . . .
6.4 Implementation Details . . . . . .

7 Challenges&Trade-offs

7.1 Retrieval Accuracy vs. Speed . . . . . .
7.2 Chunk Size vs. Context Quality . . . . . .
7.3 Cost vs. Performance . . . . . .

8 TestingStrategy 16
8.1 Testing Approach . . . . . .
8.2 Sample Queries for Testing . . . . . .
8.3 TestResults..... . . . . .

9 FutureEnhancements 18
9.1 Multi-DocumentSupport. . . . . .
9.2 FeedbackLoop . . . . . .
9.3 PersistentMemory . . . . . .
9.4 Cloud Deployment . . . . . .
9.5 TechnicalEnhancements . . . . . .

Chapter 1

Introduction

1.1 WhatisRAG?

Retrieval-Augmented Generation (RAG) is an AI architecture that combines two systems:

1. Retrieval System: A search engine that finds relevant information from a knowledge base (documents, FAQs, manuals).
2. Generation System: A Large Language Model (LLM) that generates natural language responses.

The key insight is: LLMs are excellent at generating human-like text, but they can hallucinate facts. By first retrieving verified information from a trusted knowledge base and then augmenting the LLM's prompt with that information, we get responses that are both fluent and factually grounded.

Without RAG: The LLM answers from its training data (which may be outdated or wrong).

With RAG: The LLM answers from your specific documents (accurate, up-to-date, verifiable).

1.2 WhyisRAGNeeded?

Table 1.1: Problems RAG Solves

Problem	How RAG Solves It
LLMs hallucinate facts	Responses are grounded in retrieved documents
Training data becomes stale	Knowledge base can be updated without retraining
LLMs lack domain-specific knowledge	Custom PDFs provide domain expertise
No source attribution	Retrieved chunks provide traceable sources
Expensive fine-tuning	RAG is cheaper than fine-tuning for most use cases

1.3 UseCaseOverview

This system implements a Customer Support Assistant for "TechFlow Solutions." The assistant:

- Answers FAQs about products, pricing, account management, and policies
- Provides troubleshooting steps for common technical issues
- Routes complaints and complex queries to human agents
- Shows transparency by displaying source documents used for each answer
- Escalates intelligently when it detects uncertainty or sensitive topics

Example interactions:

- ”How do I reset my password?” → RAG retrieves password reset docs → Generates step-by-step answer
- ”I’m very frustrated with your service!” → Intent: COMPLAINT → Escalated to human agent
- ”What is quantum physics?” → OUT OF SCOPE → Escalated (outside knowledge base)

Chapter 2

System Architecture Explanation

2.1 System Architecture Overview

Figure 2.1 shows the complete five-layer system architecture.

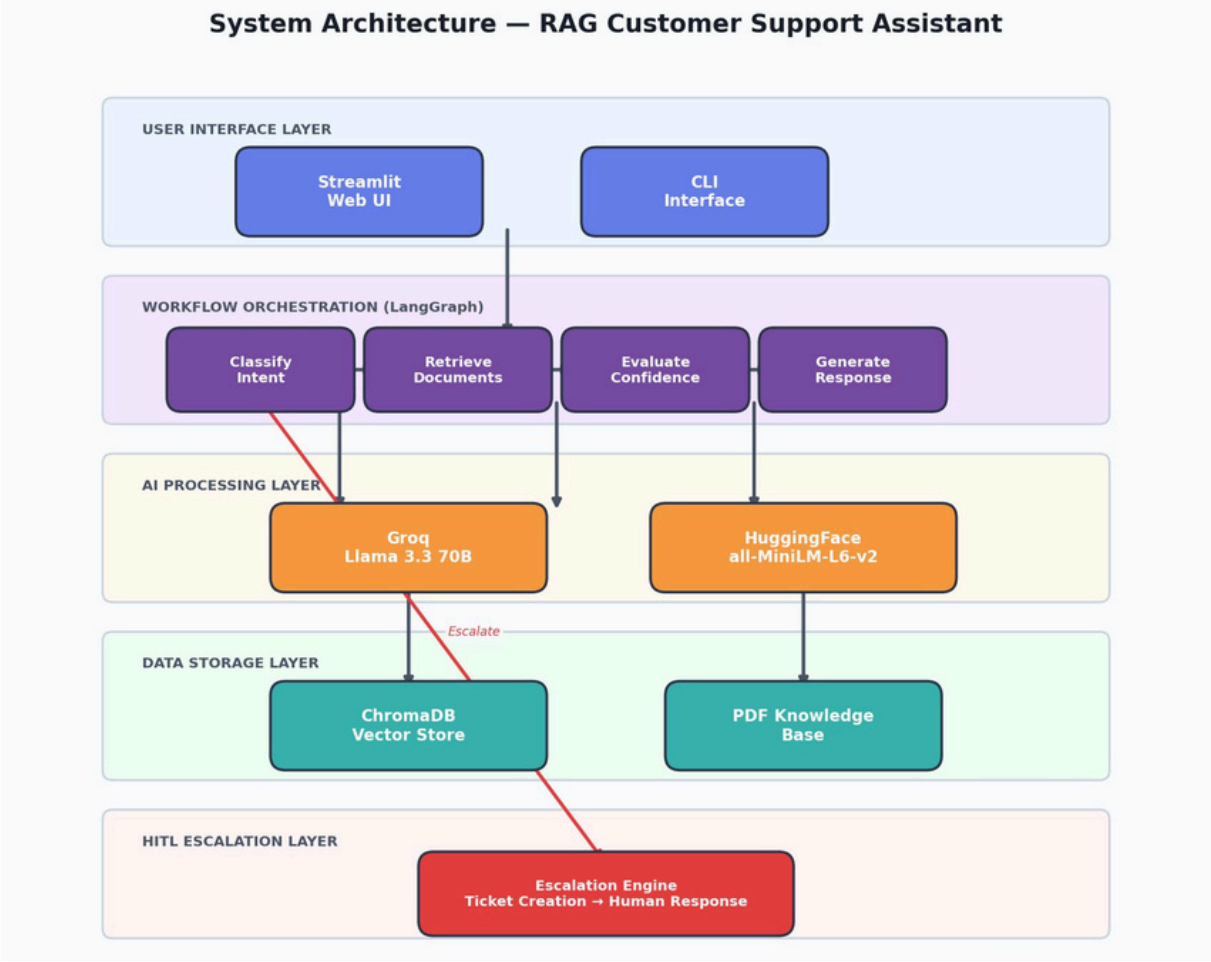

Figure 2.1: System Architecture — RAG Customer Support Assistant

2.2 Five Architecture Layers

2.2.1 Layer 1: User Interface

The Streamlit web application provides a chat interface using st.chat message and st.chat input for natural conversation, a sidebar for PDF upload and knowledge base management, an escalation panel that appears when HITL is triggered, and source document display via expandable sections. The UI maintains conversation history via st.session state, enabling multi-turn conversations within a session.

2.2.2 Layer 2: Workflow Orchestration (LangGraph)

This is the brain of the system. The LangGraph StateGraph manages the entire query lifecycle through 5 nodes and 2 conditional edges. The graph operates on a shared GraphState TypedDict, where each node reads from and writes to specific fields. This architecture ensures complete traceability, idempotent execution, and clean separation of concerns.

2.2.3 Layer 3: AI Processing

Two AI models work together:

- Groq Llama 3.3 70B (LLM): Handles intent classification and response generation at temperature=0.3 for consistent, factual outputs.
- HuggingFace Embeddings (local): Converts text into 384-dimensional vectors using all-MiniLM-L6-v2 running locally. Both queries and document chunks use the same model, ensuring compatible vector spaces.

2.2.4 Layer 4: Data Storage

ChromaDB serves as the vector database, storing document chunk embeddings alongside raw text and metadata, performing cosine similarity search, persisting data to disk, and supporting metadata filtering.

2.2.5 Layer 5: HITL System

The escalation engine monitors multiple signals (intent classification results, retrieval confidence scores, explicit user requests), creates context-rich tickets, and presents a human response interface.

Chapter 3

Design Decisions

3.1 Chunk Size Choice: 500 Characters

Decision: 500 characters per chunk with 100-character overlap (20%).

- Too small (100–200 chars): Chunks lose context — a question about password reset might match a chunk that only says "passwords" without the reset steps.
- Too large (1000+ chars): Chunks become noisy — diluting relevance by containing content about multiple unrelated topics.
- 500 chars (sweet spot): Typically captures a complete FAQ answer or troubleshooting step.

Why 20% overlap: Without overlap, a sentence spanning two chunk boundaries may be incomplete in both. With 100-char overlap, the critical sentence appears fully in at least one chunk.

3.2 Embedding Strategy

Model: HuggingFace all-MiniLM-L6-v2 (local)

- Runs entirely on your machine — zero API cost, no rate limits
- Works offline — no network latency
- 384-dimensional vectors — compact yet effective
- ~80 MB model cached locally after first download

Consistency principle: The same embedding model must be used for both document ingestion and query encoding. Mixing models would produce incompatible vector spaces and meaningless similarity scores.

3.3 Retrieval Approach: K=3, Cosine Similarity

Why cosine similarity? Standard for text embeddings, scale-invariant, built into ChromaDB as the default metric.

Why K=3?

- K=1: Too risky — might miss the best answer if the top result is slightly off-topic
- K=3: Provides redundancy — if one chunk is partial, the others fill gaps
- K=5+: Diminishing returns — more chunks add noise and slow responses

3.4 Prompt Design Logic

The system prompt establishes five key elements:

1. Identity: ”You are a helpful and professional AI Customer Support Assistant.”
2. Grounding Rule: ”Answer ONLY based on the provided context.” (prevents hallucination)
3. Uncertainty Handling: Explicit instruction for when context is insufficient
4. Tone: ”Be polite, concise, and professional.”
5. Formatting: ”Format your response with bullet points or numbered steps.”

Why temperature=0.3? Controls randomness: 0.0 = deterministic, 0.3 = slight variety without wild deviations, 1.0+ = very creative (unsuitable for factual support).

Chapter 4

Workflow Explanation

4.1 LangGraph Usage

LangGraph supports conditional branching (different paths based on runtime decisions), shared state (all nodes read/write the same state dictionary), and built-in HITL via the `interrupt()` function.

Why not a simple sequential chain? A sequential chain would process every query the same way. Customer support requires routing: complaints should skip retrieval, low- confidence retrievals should escalate, and out-of-scope queries need different handling. LangGraph's conditional edges make this natural and explicit.

4.2 NodeResponsibilities

Table 4.1: Node Responsibilities

Node	Resp onsibility	LLM?	DB?
classify_intent	Determine what the user wants	Yes	No
retrieve_documents	Find relevant KB chunks	No	Yes
evaluate_confidence	Score retrieval quality	No	No
generate_response	Create the final answer	Yes	No
escalate_to_human	Create ticket + escalation msg	No	No

4.3 State Transitions

4.3.1 NormalFlow(FAQQuery)

Listing 4.1: Normal State Transition Example

```
# Initial
{query: "How to reset password?", intent: "", response: ""}

# After classify_intent
{intent: "GENERAL_QUERY", needs_escalation:false}

# After retrieve_documents
{retrieved_docs: [(doc1, 0.89), ...], context: "...", sources: [...]}

# After evaluate_confidence
{retrieval_confidence: 0.89, needs_escalation: false}

# After generate_response
{response: "To reset your password, follow these steps: 1) Go..."}
```

4.3.2 Escalation Flow (Complaint)

Listing 4.2: Escalation State Transition Example

```
# Initial
{query: "I'm furious! Your service is terrible!", intent: ""}

# After classify_intent
{intent: "COMPLAINT", needs_escalation: true}
# --> SKIP retrieve_documents andevaluate_confidence

# After escalate_to_human
{response: "I'm sorry to hear... escalating to humanagent.",
escalation_ticket: {ticket_id: "ESC-4291", ...}}
```

4.4 LangGraph Workflow Diagram

Figure 4.1 shows the full StateGraph workflow with all conditional routing paths.

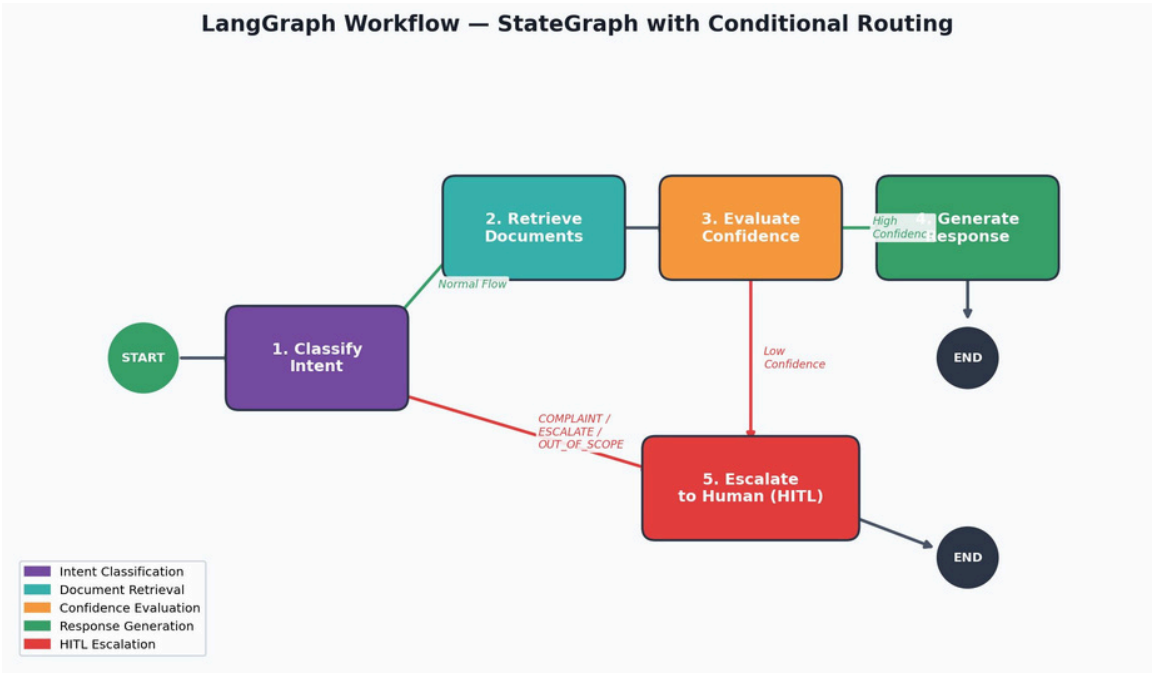

Figure 4.1: LangGraph Workflow — StateGraph with Conditional Routing

Chapter 5

Conditional Logic

5.1 Intent Detection

The LLM classifies each query into exactly one of six categories:

Table 5.1: Intent Categories

Intent	Examples	Action
GENERAL QUERY	”What are your plans?”, ”How to create account?”	Retrieve → Generate
TECHNICAL ISSUE	”My login isn’t working”, ”Video call lagging”	Retrieve → Generate
BILLING	”How do I get a refund?”, ”What’s my invoice?”	Retrieve → Generate
COMPLAINT	”I’m very frustrated!”, ”Your service is awful”	Escalate immediately
ESCALATE	”Let me talk to a human”, ”I need a manager”	Escalate immediately
OUT OF SCOPE	”What’s the weather?”, ”Explain quantum physics”	Escalate immediately

Fallback mechanism: If the LLM returns an unrecognised category, keyword-based fuzzy matching attempts to map it to the closest valid category. If all else fails, it defaults to GENERAL QUERY.

5.2 Routing Decisions

Listing 5.1: Routing Functions

```
# Decision Point 1: After Intent Classification
def route_after_intent(state):
    if state["needs_escalation"]:    # COMPLAINT/ESCALATE/OOS
        return "escalate_to_human"
    else:
        return "retrieve_documents"

# Decision Point 2: After Confidence Evaluation
def route_after_confidence(state):
    if state["needs_escalation"]:    # confidence < 0.4 or no docs
        return "escalate_to_human"
    else:
        return "generate_response"
```

Why two decision points?

1. First checkpoint (intent): Catchesobviouslynon-retrievablequeriesearly,saving unnecessary vector search and LLM costs.
2. Second checkpoint (confidence): Catches cases where the query seems answerable but the knowledge base doesn't have good matches.

5.3 DecisionTree

Figure 5.1 shows the complete conditional routing decision tree.

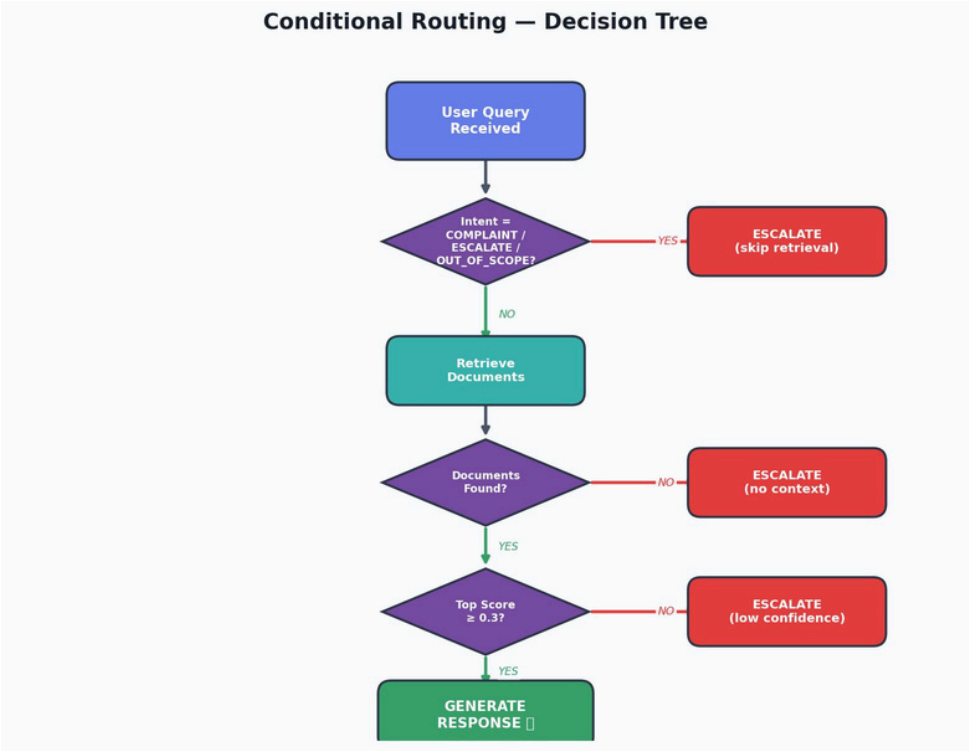

Figure 5.1: Conditional Routing — Decision Tree

Chapter 6

HITL Implementation

6.1 Role of Human Intervention

The HITL system serves as a safety net for the AI. Human intervention is essential for:

- Emotional situations: Complaints require empathy that AI may not reliably deliver
- Edge cases: Queries outside the training data or knowledge base
- High-stakes decisions: Billing disputes, account changes, or legal matters
- Quality assurance: Catching potential hallucinations or inaccurate responses

6.2 Benefits and Limitations of HITL

Table 6.1: HITL Benefits

Benefit	Explanation
Customer satisfaction	Frustrated customers get human attention
Risk mitigation	AI doesn't provide wrong answers to sensitive queries
Continuous improvement	Human responses can improve the knowledge base
Trust building	Transparency about AI limitations increases user trust
Legal compliance	Some regulations require human oversight

Table 6.2: HITL Limitations

Limitation	Impact
Latency	Human response takes minutes/hours vs. AI's seconds
Availability	Humans aren't 24/7 unless specifically staffed
Cost	Human agents are more expensive than AI processing
Consistency	Different agents may give different answers
Scalability	Limited capacity; can't handle sudden query spikes

6.3 HITLEscalationFlow

Figure 6.1 illustrates the complete escalation process.

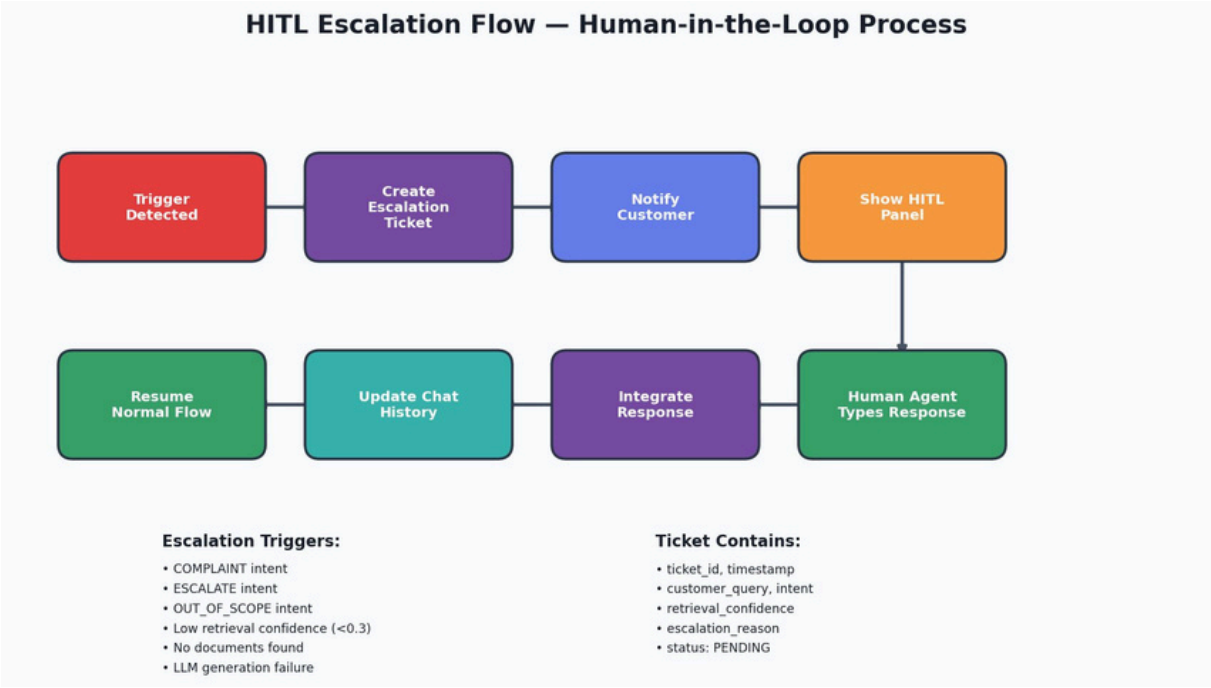

Figure 6.1: HITL Escalation Flow — Human-in-the-Loop Process

6.4 Implementation Details

Listing 6.1: Multi-Factor Escalation Check

```
def should_escalate(state):
    return any([
        state["intent"] in ["COMPLAINT", "ESCALATE",
                            "OUT_OF_SCOPE"],
        state["retrieval_confidence"] < 0.4,
        state.get("needs_escalation", False),
        len(state.get("retrieved_docs", [])) == 0,
    ])
```

Listing 6.2: Escalation Ticket Structure

```
{
    "ticket_id": "ESC-4291", "timestamp":
    "2026-04-22 10:15:30", cancel
    "customer_query": "I want to and I'm very frustrated!",
    "detected_intent": "COMPLAINT",
    "retrieval_confidence": 0.0, # N/A (skipped)
    "escalation_reason": "Customer has expressed dissatisfaction.",
    "retrieved_context": "Summary of any retrieved docs",
    "status": "PENDING_HUMAN_REVIEW",
}
```

Chapter 7

Challenges & Trade-offs

7.1 Retrieval Accuracy vs. Speed

Table 7.1: Retrieval Strategy Comparison

Approach	Accuracy	Sp eed	Our Choice
Exhaustivesearch	Highest	Slowest	No
Approximate NN (HNSW)	High	Fast	
Keywordsearch(BM25)	Medium	Fastest	Yes
Hybrid (vector + keyword)	Highest	Mo derate	No
			Future enhancement

7.2 Chunk Size vs. Context Quality

Table 7.2: Chunk Size Trade-offs

Chunk Size	Precision	Context	Embedding Quality
100 chars	High	Low (fragments)	Poor (too short)
300 chars	High	Medium	Good
500 chars	Med-High	Good	Good (chosen)
1000 chars	Medium	High	Good but noisy
Full page	Low	Very High	Noisy

7.3 Cost vs. Performance

Table 7.3: Cost vs Performance Trade-offs

Comp onent	Free Tier	Paid Option	Difference
LLM	Groq Llama 3.3 70B	GPT-4o	GPT-4o better for nuance
Embeddings	HuggingFace local	OpenAI text-embedding-3	Marginal improvement
Vector DB	ChromaDB (local)	Pinecone	Pinecone better at scale
Hosting	Lo cal	Cloud VM	Required for pro duction

Chapter 8

Testing Strategy

8.1 Testing Approach

The testing strategy follows a pyramid approach: many unit tests, some integration tests, and a few end-to-end tests.
Unit Tests (27 tests, all passing):

- `test_document_processor.py` (8 tests): PDF loading, chunking, metadata
- `test_retriever.py` (6 tests): Confidence scoring, context formatting
- `test_intent_classifier.py` (8 tests): Fuzzy matching, escalation reasons
- `test_graph_workflow.py` (5 tests): Routing logic, state schema

8.2 Sample Queries for Testing

Table 8.1: Test Query Matrix

Query	Expected Intent	Expected Behaviour
"How do I reset my password?"	GENERAL_QUERY	RAG retrieval → confident answer
"What subscription plans do you offer?"	GENERAL_QUERY	RAG retrieval → plan details
"My video calls keep dropping"	TECHNICAL_ISSUE	RAG → troubleshooting steps
"Iwantarefundforlastmonth"	BILLING	RAG → refund policy
"Yourserviceisabsolutelyterrible!" "Letmespeaktoamanager"	COMPLAINT	ImmediateHITLescalation
	ESCALATE	ImmediateHITLescalation
"What is the capital of France?"	OUT_OF_SCOPE	Escalation (outside KB)
"HowdoIexportmydata?"	GENERALQUERY	RAG → export instructions

8.3 Test Results

Listing 8.1: Test Suite Results

```
============================================================
test session results
============================================================
```

```
collected  27 items

tests/test_document_processor.py  ........          [29%]
tests/test_graph_workflow.py      .....             [48%]
tests/test_intent_classifier.py   ........          [77%]
tests/test_retriever.py           ......            [100%]

============================================================
27  passed in 2.72s
============================================================
```

Chapter 9

Future Enhancements

9.1 Multi-DocumentSupport

- Current: Single PDF knowledge base
- Future: Support multiple PDFs with per-document metadata filtering
- Implementation: Add a source collection field to metadata; allow users to query specific document sets

9.2 Feedback Loop

- Current: No learning from interactions
- Future: Track user satisfaction (thumbs up/down on responses)
- Implementation: Store (query, response, feedback) triplets; use positive examples for few-shot prompts; use negative examples to identify knowledge gaps

9.3 Persistent Memory

- Current: Session-based chat history (lost on refresh)
- Future: Persistent conversation memory across sessions using LangGraph's `PostgresSaver` checkpointer, user profile storage, and conversation summaries for long-term context

9.4 Cloud Deployment

- Containerise with Docker
- Deploy to cloud (AWS ECS, GCP Cloud Run, or Streamlit Cloud)
- Add authentication (OAuth)
- Set up monitoring (LangSmith + cloud metrics)
- Configure auto-scaling based on query volume

9.5 Technical Enhancements

- Hybrid Search: Combine vector search with BM25 keyword search for better retrieval
- Re-ranking: Add a cross-encoder re-ranker to improve retrieved document relevance
- Streaming Responses: Stream LLM output token-by-token for better UX

- **Multi-language:** Support queries in multiple languages
- **Analytics Dashboard:** Track query patterns, escalation rates, and response quality
- **Automated KB Updates:** Detect outdated information and suggest updates